NAND to Tetris

Boolean Logic & Boolean Arithmetic

Project 1 & Project 2 · HDL Implementation

Name: **RAVINDER** | Registration No.: **2501420018**

K.R. Mangalam University, Gurugram, Haryana

Introduction

The **NAND to Tetris** course (nand2tetris.org) guides students from a single NAND gate all the way to a working general-purpose computer and modern software hierarchy. This assignment covers **Project 1 (Boolean Logic)** and **Project 2 (Boolean Arithmetic)**, implemented entirely in the Hardware Description Language (HDL) provided by the Nand2Tetris Software Suite.

PROJECT 1 — Boolean Logic

In Project 1 every logic gate is built using only the primitive **NAND** gate. Starting from NAND, we derive NOT, AND, OR, XOR, MUX, DMUX, and their multi-bit / multi-way variants.

1.1 Not Gate

Inverts a single bit. Built by feeding the same input into both NAND inputs.

in	out
0	1
1	0

```
// Not.hdl // Implements: out = NOT in

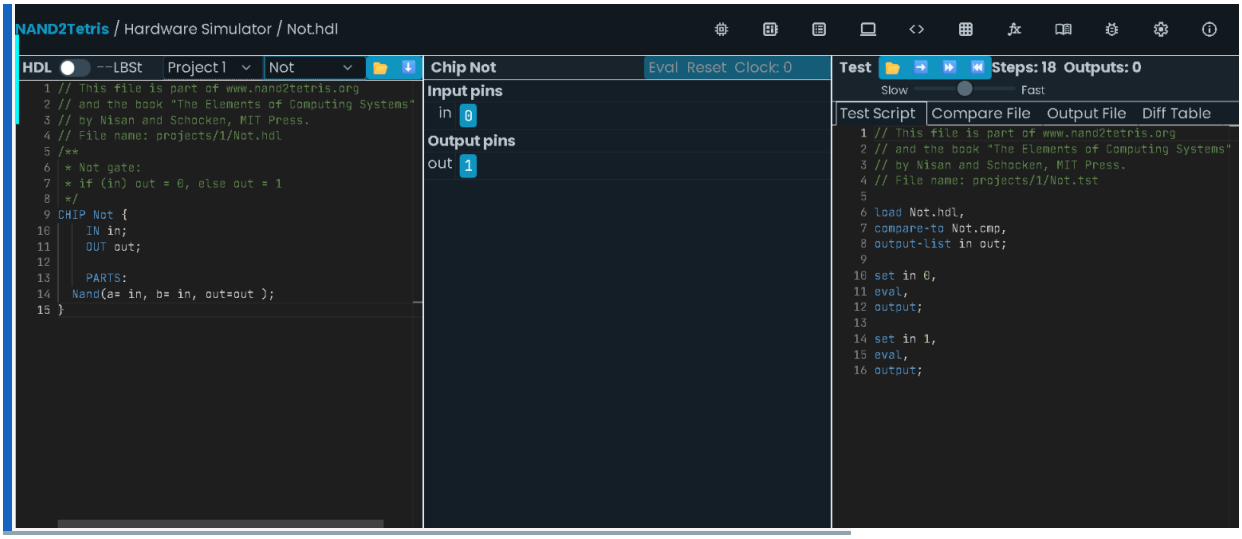
CHIP Not {

IN in;

OUT out;

PARTS: Nand(a=in, b=in, out=out);

}
```



1.2 And Gate

Returns 1 only when both inputs are 1. NAND followed by NOT.

a	b	out
0	0	0
0	1	0
1	0	0
1	1	1

```
// And.hdl // Implements: out = a AND b

CHIP And {

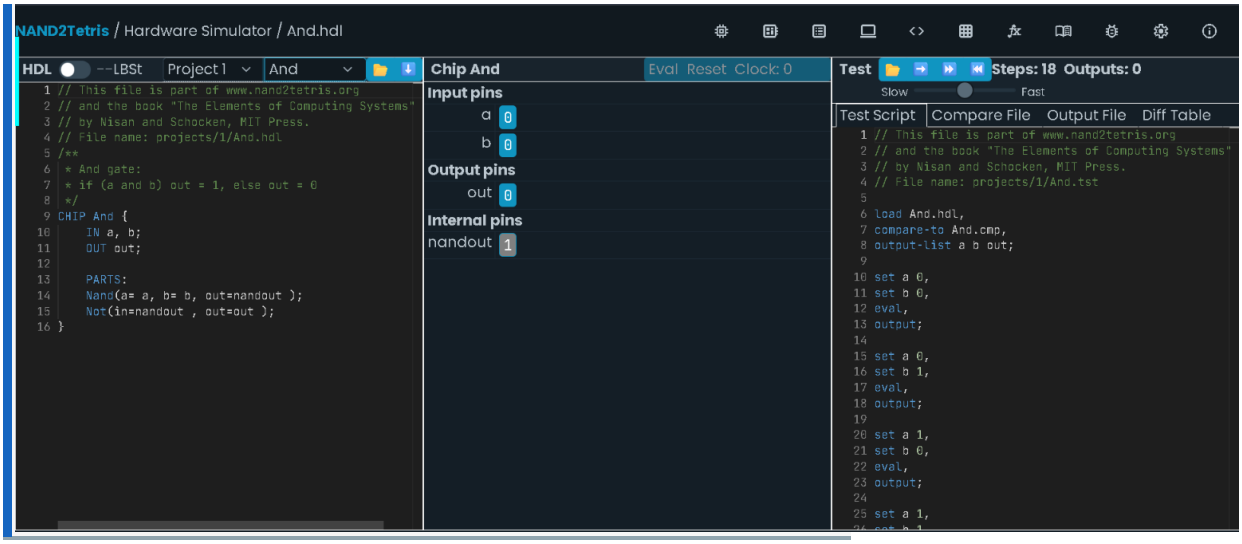
IN a, b;

OUT out;

PARTS:

Nand(a=a, b=b, out=nandOut); Not(in=nandOut, out=out);

}
```



1.3 Or Gate

Returns 1 when at least one input is 1. Uses De Morgan's theorem: a OR b = NOT(NOT a AND NOT b).

a	b	out
0	0	0
0	1	1
1	0	1
1	1	1

```
// Or.hdl // Implements: out =

a OR b

CHIP Or {

IN a, b;

OUT out;

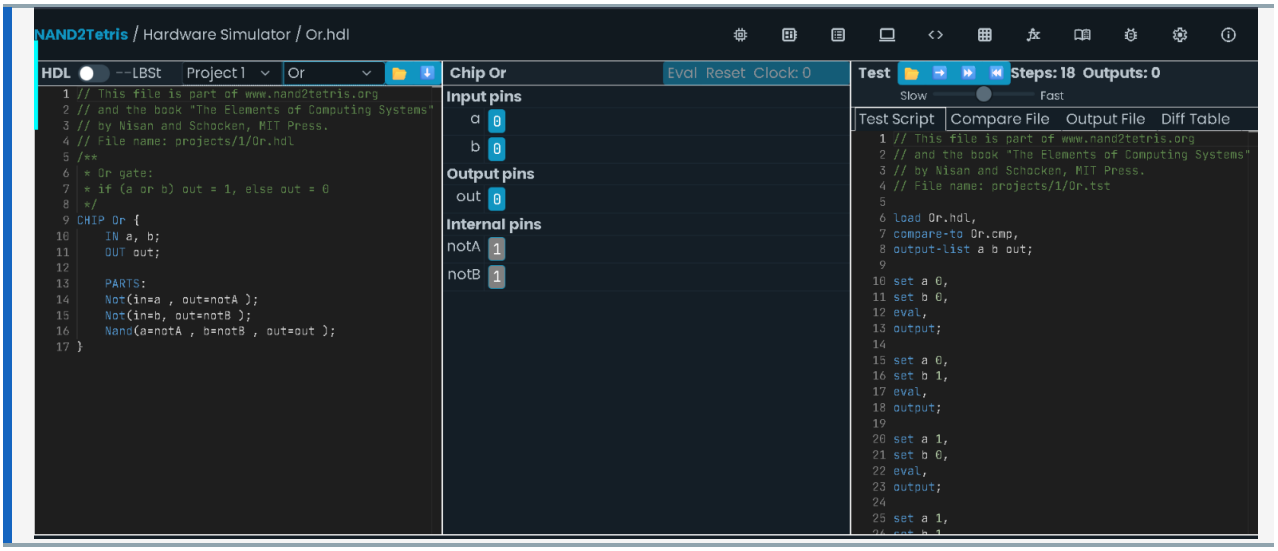
PARTS:

Not(in=a, out=notA);

Not(in=b, out=notB); Nand(a=notA,

b=notB, out=out);

}
```



1.4 Xor Gate

Returns 1 when exactly one of the inputs is 1 (exclusive OR).

a	b	out
0	0	0
0	1	1
1	0	1
1	1	0

```
// Xor.hdl // Implements: out =

(a XOR b)

CHIP Xor {

IN a, b;

OUT out;

PARTS:

Not(in=a, out=notA);

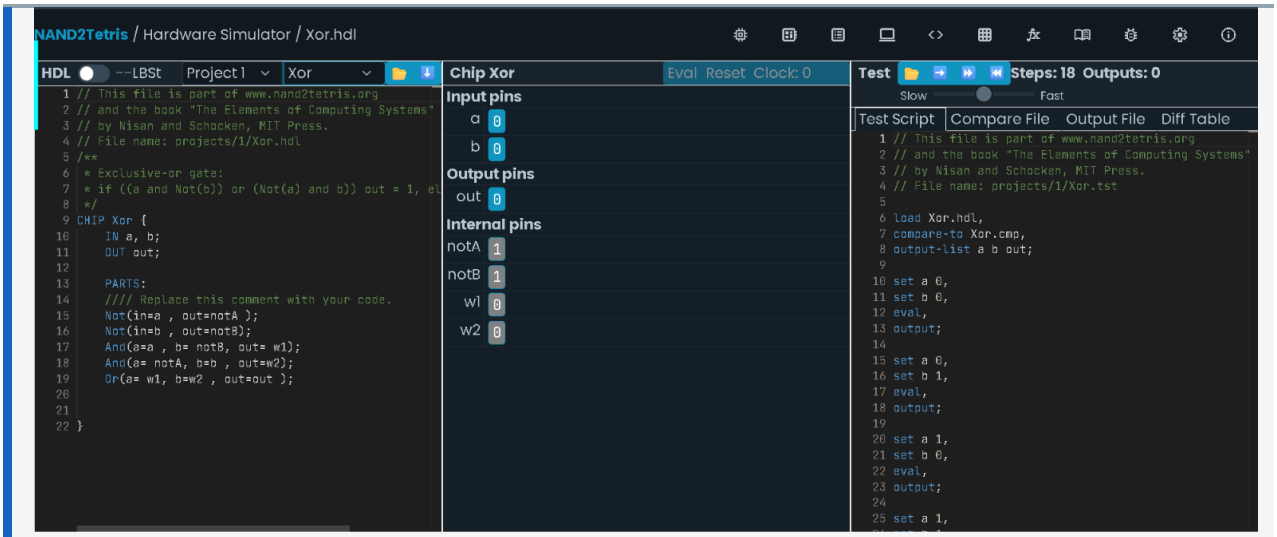
Not(in=b, out=notB);

And(a=a, b=notB, out=aAndNotB);

And(a=notA,b=b, out=notAAndB); Or(a=aAndNotB,

b=notAAndB, out=out);

}
```



1.5 Mux (Multiplexer)

Selects between two inputs using a selector bit: if sel=0 output is a, if sel=1 output is b.

a	b	sel	out
0	0	0	0
0	1	0	0
1	0	0	1
1	1	0	1
0	0	1	0
0	1	1	1
1	0	1	0
1	1	1	1

```
// Mux.hdl // Implements: if (sel==0) out=a else out=b

CHIP Mux {

IN a, b, sel;

OUT out;

PARTS:

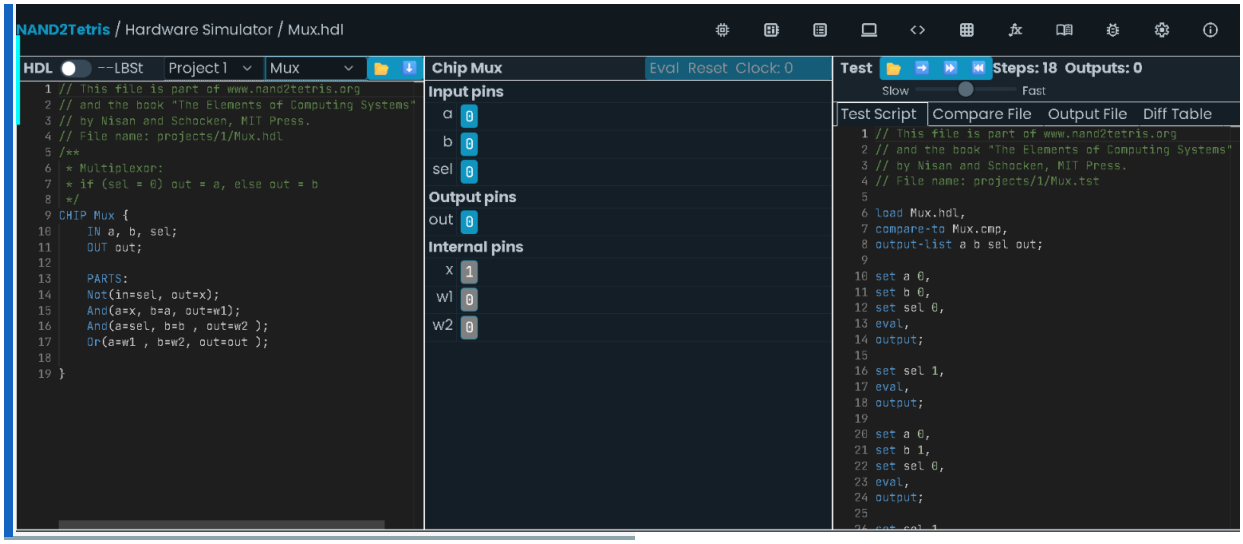
Not(in=sel, out=notSel);

And(a=a, b=notSel, out=aAndNotSel);
```

```
And(a=b, b=sel, out=bAndSel);

Or(a=aAndNotSel, b=bAndSel, out=out);

}
```



1.6 DMux (Demultiplexer)

Routes one input to one of two outputs based on the selector: if sel=0 then {a=in, b=0}, else {a=0, b=in}.

```
// DMux.hdl // Implements: if (sel==0) {a=in; b=0} else

{a=0; b=in}

CHIP DMux {

IN in, sel;

OUT a, b;

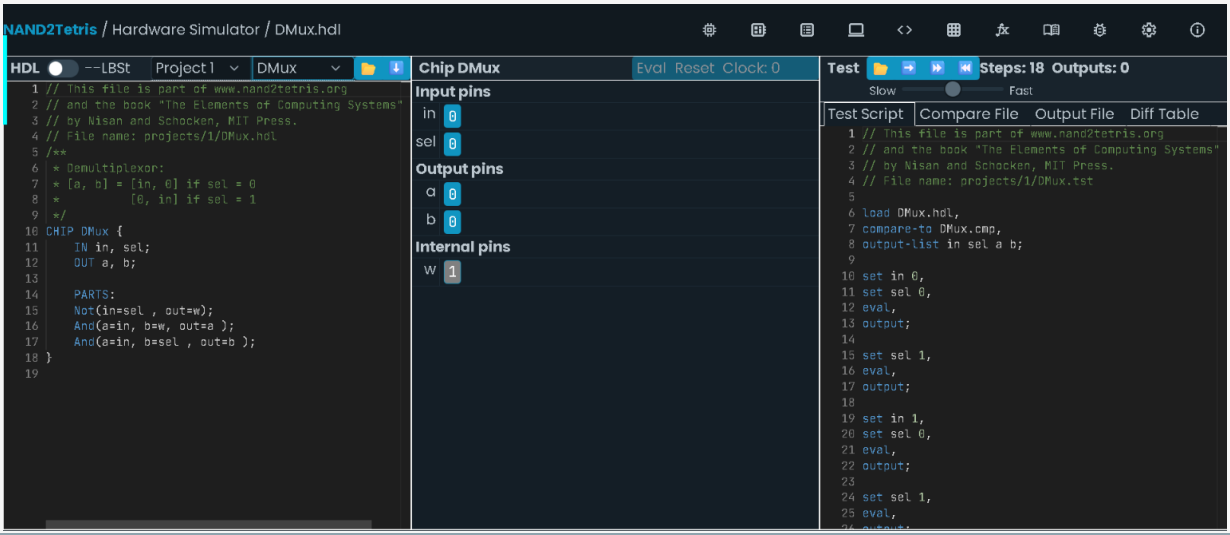
PARTS:

Not(in=sel, out=notSel);

And(a=in, b=notSel, out=a); And(a=in,

b=sel, out=b);

}
```



1.7 Not16 (16-bit Not)

Applies the Not operation to each of the 16 bits of the input bus independently.

```
// Not16.hdl // Implements: for i in 0..15: out[i] = NOT in[i]
```

```
CHIP Not16 {

IN in[16];

OUT out[16];

PARTS:

Not(in=in[0], out=out[0]);

Not(in=in[1], out=out[1]);

Not(in=in[2], out=out[2]);

Not(in=in[3], out=out[3]);

Not(in=in[4], out=out[4]);

Not(in=in[5], out=out[5]);

Not(in=in[6], out=out[6]);

Not(in=in[7], out=out[7]);

Not(in=in[8], out=out[8]);

Not(in=in[9], out=out[9]);

Not(in=in[10], out=out[10]);

Not(in=in[11], out=out[11]);

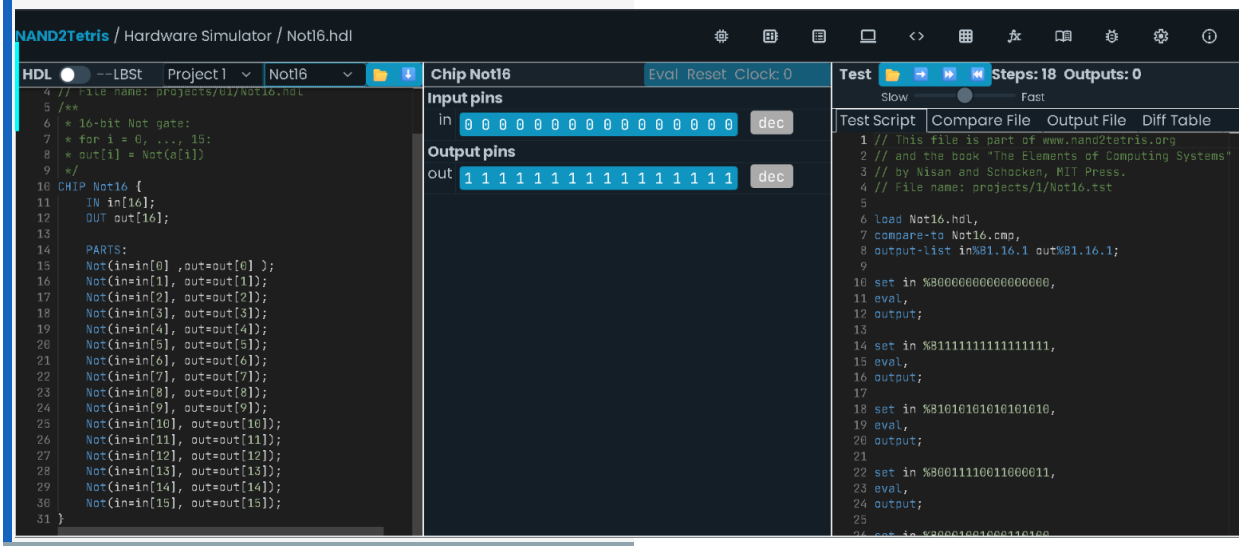
Not(in=in[12], out=out[12]);
```

```
Not(in=in[13], out=out[13]);

Not(in=in[14], out=out[14]); Not(in=in[15],

out=out[15]);

}
```



1.8 And16 (16-bit And)

Applies bitwise AND across two 16-bit buses.

```
// And16.hdl // Implements: for i in 0..15: out[i] =

a[i] AND b[i]

CHIP And16 {

IN a[16], b[16];

OUT out[16];

PARTS:

And(a=a[0], b=b[0], out=out[0]);

And(a=a[1], b=b[1], out=out[1]);

And(a=a[2], b=b[2], out=out[2]);

And(a=a[3], b=b[3], out=out[3]);

And(a=a[4], b=b[4], out=out[4]);

And(a=a[5], b=b[5], out=out[5]);

And(a=a[6], b=b[6], out=out[6]);

And(a=a[7], b=b[7], out=out[7]);

And(a=a[8], b=b[8], out=out[8]);

And(a=a[9], b=b[9], out=out[9]);

And(a=a[10], b=b[10], out=out[10]);

And(a=a[11], b=b[11], out=out[11]);

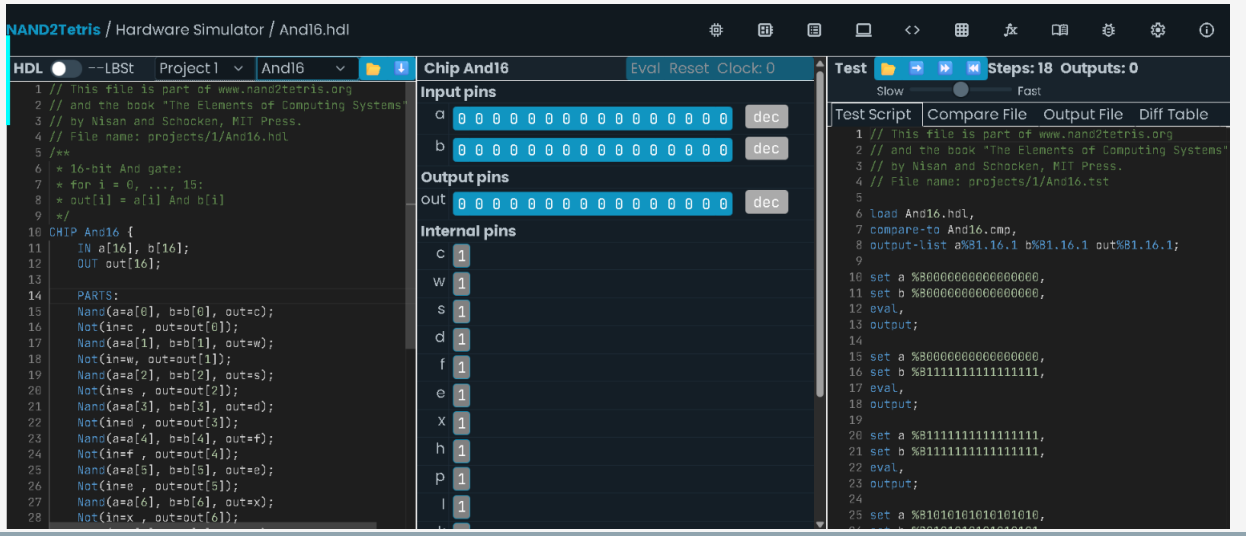
And(a=a[12], b=b[12], out=out[12]);

And(a=a[13], b=b[13], out=out[13]);

And(a=a[14], b=b[14], out=out[14]); And(a=a[15],

b=b[15], out=out[15]);

}
```



1.9 Or16 (16-bit Or)

Applies bitwise OR across two 16-bit buses.

```
// Or16.hdl // Implements: for i in 0..15: out[i] = a[i] OR b[i]

CHIP Or16 {

IN a[16], b[16];
```

```
OUT out[16];
```

```
PARTS:

Or(a=a[0], b=b[0], out=out[0]);

Or(a=a[1], b=b[1], out=out[1]);

Or(a=a[2], b=b[2], out=out[2]);

Or(a=a[3], b=b[3], out=out[3]);

Or(a=a[4], b=b[4], out=out[4]);

Or(a=a[5], b=b[5], out=out[5]);

Or(a=a[6], b=b[6], out=out[6]);

Or(a=a[7], b=b[7], out=out[7]);

Or(a=a[8], b=b[8], out=out[8]);

Or(a=a[9], b=b[9], out=out[9]);

Or(a=a[10], b=b[10], out=out[10]);

Or(a=a[11], b=b[11], out=out[11]);

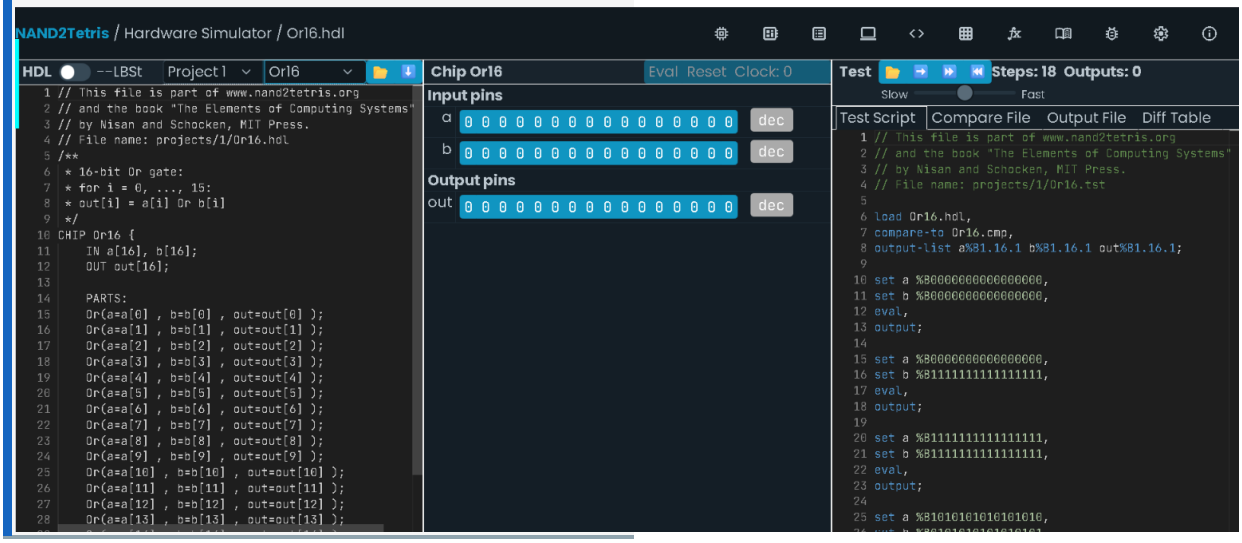
Or(a=a[12], b=b[12], out=out[12]);

Or(a=a[13], b=b[13], out=out[13]);

Or(a=a[14], b=b[14], out=out[14]);

Or(a=a[15], b=b[15], out=out[15]);

}
```



1.10 Mux16 (16-bit Mux)

Selects between two 16-bit buses using a single selector bit.

```
// Mux16.hdl // Implements: if (sel==0) out=a else out=b (16-bit wide)

CHIP Mux16 {

IN a[16], b[16], sel;

OUT out[16];
```

```
PARTS:

Mux(a=a[0], b=b[0], sel=sel, out=out[0]);

Mux(a=a[1], b=b[1], sel=sel, out=out[1]);

Mux(a=a[2], b=b[2], sel=sel, out=out[2]);

Mux(a=a[3], b=b[3], sel=sel, out=out[3]);

Mux(a=a[4], b=b[4], sel=sel, out=out[4]);

Mux(a=a[5], b=b[5], sel=sel, out=out[5]);

Mux(a=a[6], b=b[6], sel=sel, out=out[6]);

Mux(a=a[7], b=b[7], sel=sel, out=out[7]);

Mux(a=a[8], b=b[8], sel=sel, out=out[8]);

Mux(a=a[9], b=b[9], sel=sel, out=out[9]);

Mux(a=a[10], b=b[10], sel=sel, out=out[10]);
```

```
Mux(a=a[11], b=b[11], sel=sel, out=out[11]);

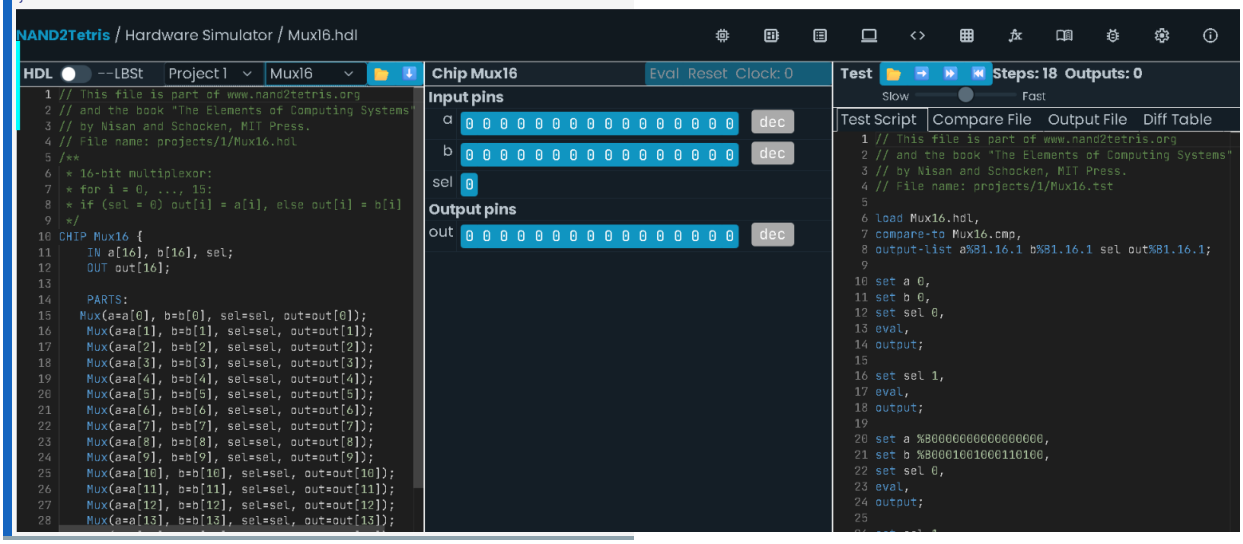
Mux(a=a[12], b=b[12], sel=sel, out=out[12]);

Mux(a=a[13], b=b[13], sel=sel, out=out[13]);

Mux(a=a[14], b=b[14], sel=sel, out=out[14]);

Mux(a=a[15], b=b[15], sel=sel, out=out[15]);

}
```



1.11 Or8Way

Returns 1 if any of the 8 input bits is 1, using a tree of Or gates.

```
// Or8Way.hdl // Implements: out = OR(in[0], in[1], ...,

in[7])

CHIP Or8Way {

IN in[8];

OUT out;

PARTS:

Or(a=in[0], b=in[1], out=or01);
```

```
Or(a=in[2], b=in[3], out=or23);

Or(a=in[4], b=in[5], out=or45);

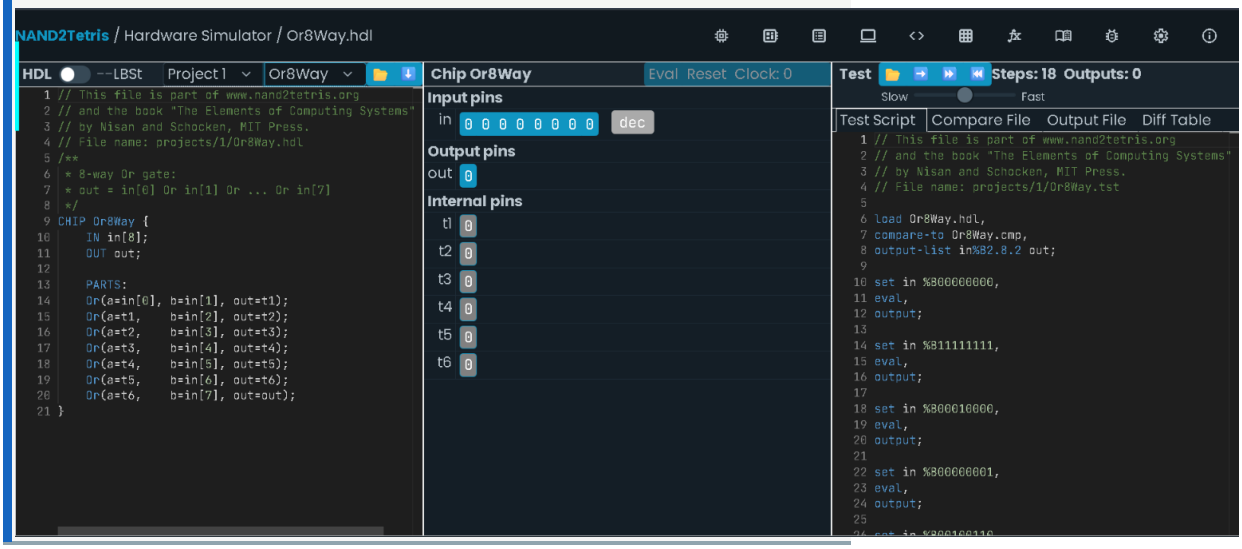
Or(a=in[6], b=in[7], out=or67);

Or(a=or01, b=or23, out=or0123);

Or(a=or45, b=or67, out=or4567);

Or(a=or0123,b=or4567,out=out);

}
```



1.12 Mux4Way16

Selects one of four 16-bit inputs using a 2-bit selector.

```
// Mux4Way16.hdl // Implements: out = a|b|c|d where sel

chooses which

CHIP Mux4Way16 { IN a[16], b[16], c[16],

d[16], sel[2];

OUT out[16];

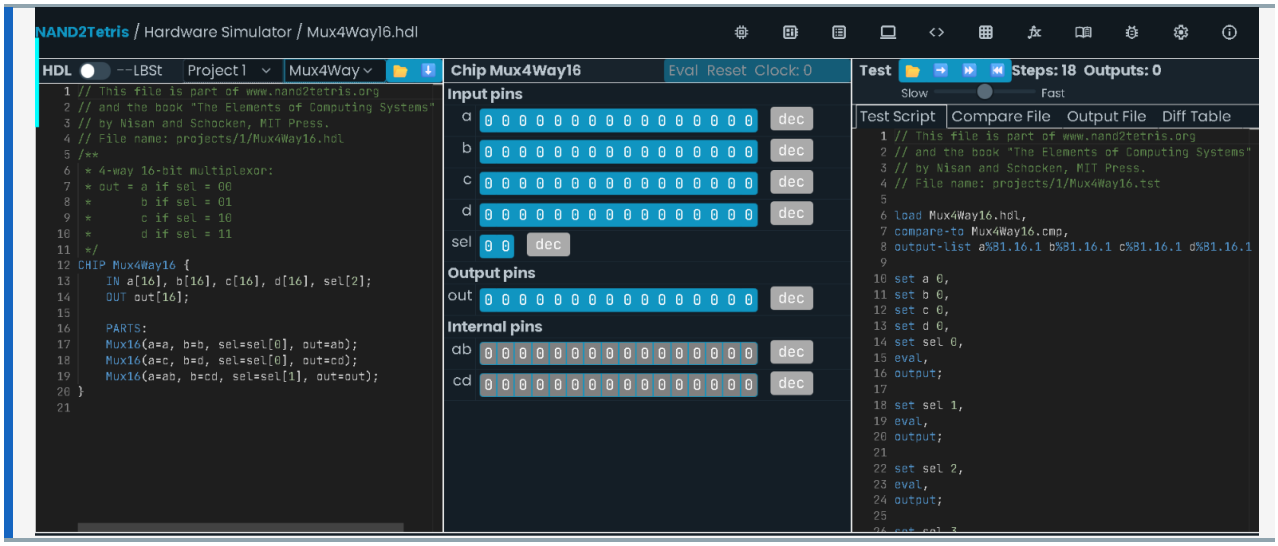
PARTS:

Mux16(a=a, b=b, sel=sel[0], out=ab);

Mux16(a=c, b=d, sel=sel[0], out=cd); Mux16(a=ab,

b=cd, sel=sel[1], out=out);

}
```



1.13 Mux8Way16

Selects one of eight 16-bit inputs using a 3-bit selector.

```
// Mux8Way16.hdl // Implements: out = one-of-{a..h}

based on sel[0..2]

CHIP Mux8Way16 { IN

a[16],b[16],c[16],d[16],

e[16],f[16],g[16],h[16],

sel[3]; OUT out[16];

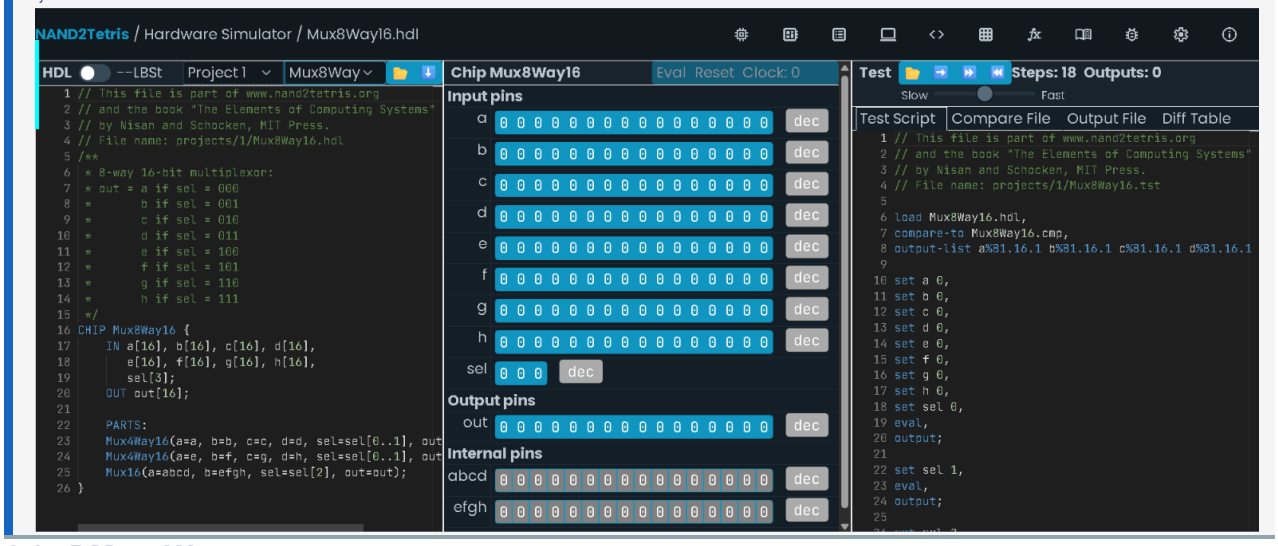
PARTS:

Mux4Way16(a=a, b=b, c=c, d=d, sel=sel[0..1], out=lo);

Mux4Way16(a=e, b=f, c=g, d=h, sel=sel[0..1], out=hi); Mux16(a=lo,

b=hi, sel=sel[2], out=out);

}
```



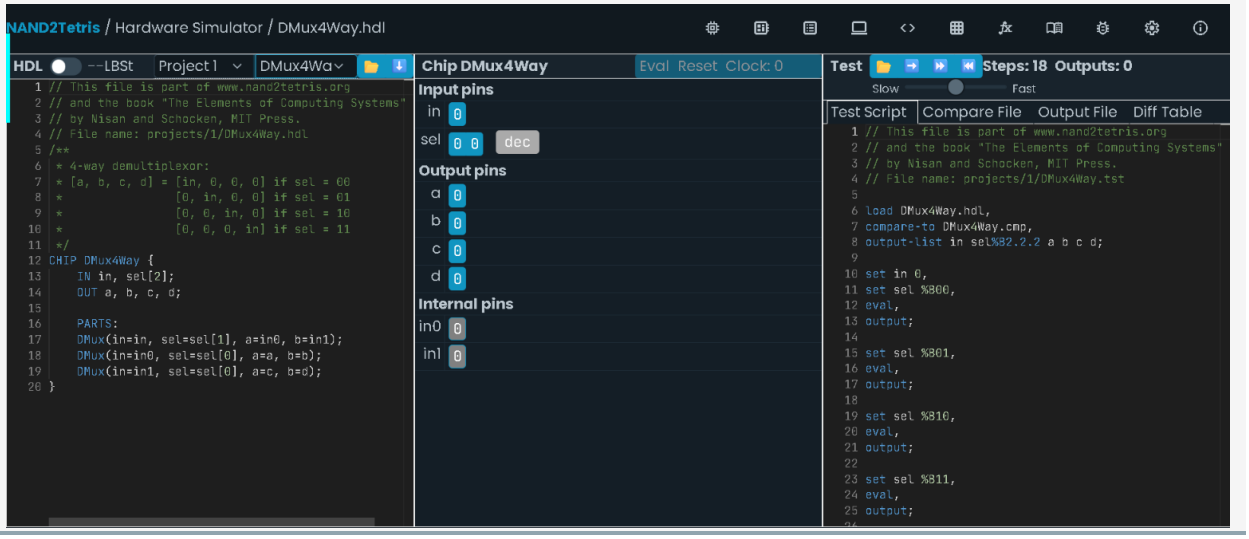
1.14 DMux4Way

Routes one input to one of four outputs using a 2-bit selector.

```
// DMux4Way.hdl // Implements: {a,b,c,d} where exactly one equals in
based on sel

CHIP DMux4Way {
IN in, sel[2];
OUT a, b, c, d;

PARTS:
DMux(in=in, sel=sel[1], a=lo, b=hi);
DMux(in=lo, sel=sel[0], a=a, b=b); DMux(in=hi,
sel=sel[0], a=c, b=d);
}
```



1.15 DMux8Way

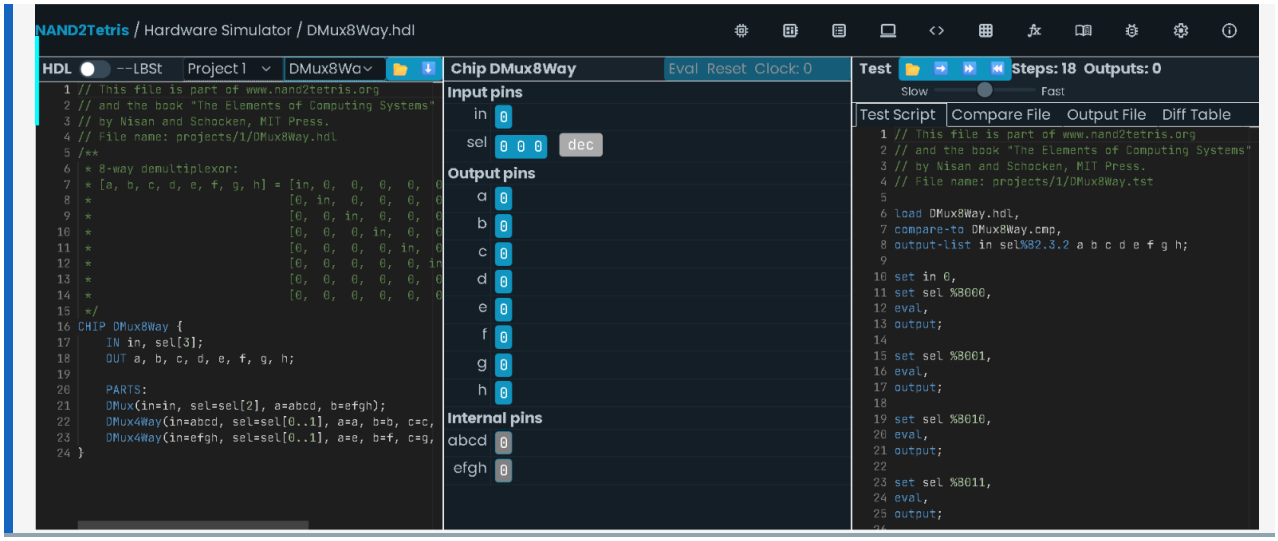
Routes one input to one of eight outputs using a 3-bit selector.

```
// DMux8Way.hdl // Implements: {a..h} where exactly one equals in
based on sel[0..2]

CHIP DMux8Way {
IN in, sel[3]; OUT a, b,
c, d, e, f, g, h;

PARTS:
DMux(in=in, sel=sel[2], a=lo, b=hi);
DMux4Way(in=lo, sel=sel[0..1], a=a, b=b, c=c, d=d);
```

```
DMux4Way(in=hi, sel=sel[0..1], a=e, b=f, c=g, d=h);
}
```


NAND2Tetris / Hardware Simulator / DMux8Way.hdl
HDL --LBSt Project 1 DMux8Wa
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/1/DMux8Way.hdl
/**
* 8-way demultiplexor:
* [a, b, c, d, e, f, g, h] = [in, 0, 0, 0, 0,
* [0, in, 0, 0, 0,
* [0, 0, in, 0, 0,
* [0, 0, 0, in, 0,
* [0, 0, 0, 0, in,
* [0, 0, 0, 0, 0, in
* [0, 0, 0, 0, 0,
* [0, 0, 0, 0, 0,
*/
CHIP DMux8Way {
IN in, sel[3];
OUT a, b, c, d, e, f, g, h;
PARTS:
DMux(in=in, sel=sel[2], a=abcd, b=efgh);
DMux4Way(in=abcd, sel=sel[0..1], a=a, b=b, c=c,
DMux4Way(in=efgh, sel=sel[0..1], a=e, b=f, c=g,
}
Chip DMux8Way
Eval Reset Clock: 0
Input pins
in 0
sel 0 0 0 dec
Output pins
a 0
b 0
c 0
d 0
e 0
f 0
g 0
h 0
Internal pins
abcd 0
efgh 0
Test
Steps: 18 Outputs: 0
Slow Fast
Test Script
Compare File
Output File
Diff Table
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/1/DMux8Way.tst
load DMux8Way.hdl,
compare-to DMux8Way.cmp,
output-list in sel%B2.3.2 a b c d e f g h;
set in 0,
set sel %B000,
eval,
output;
set sel %B001,
eval,
output;
set sel %B010,
eval,
output;
set sel %B011,
eval,
output;

PROJECT 2 — Boolean Arithmetic

Project 2 builds adder circuits and the Arithmetic Logic Unit (ALU). All chips are constructed from the gates implemented in Project 1. **Note: Inc16 is intentionally excluded from this submission.**

2.1 HalfAdder

Adds two single bits and produces a sum and carry. The sum is XOR of the inputs; the carry is AND of the inputs.

a	b	sum	carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

```
// HalfAdder.hdl // Implements: {sum, carry} = a + b

(single-bit addition)

CHIP HalfAdder {

IN a, b;

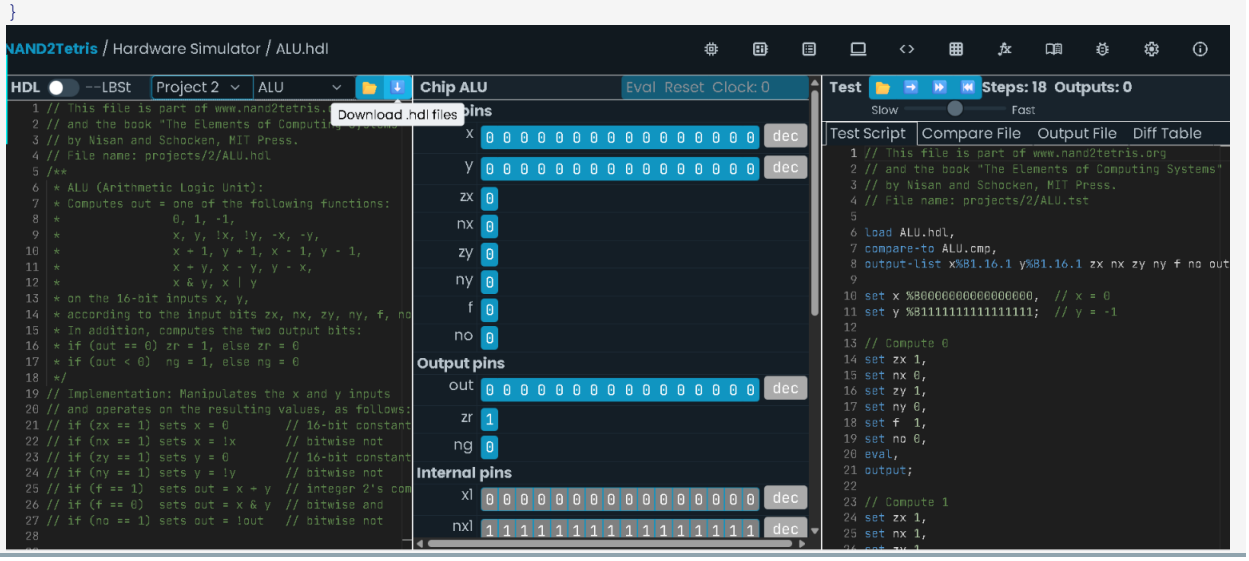
OUT sum, carry;

PARTS:

Xor(a=a, b=b, out=sum); And(a=a,

b=b, out=carry);

}
```



2.2 FullAdder

Adds three single bits (a, b, and carry-in) to produce a sum and carry-out.

A	b	c	sum	carry
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0

0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

```
// FullAdder.hdl

// Implements: {sum, carry} = a + b + c (single-bit, three-input)
```

```
CHIP FullAdder {

IN a, b, c;

OUT sum, carry;

PARTS:

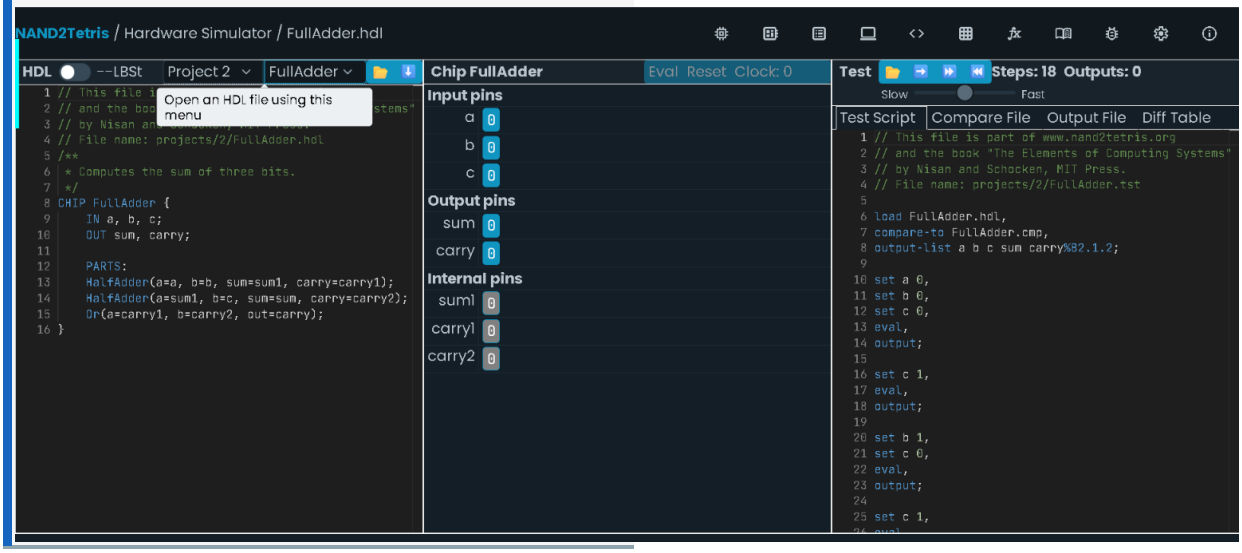
HalfAdder(a=a, b=b, sum=sum1, carry=carry1);

HalfAdder(a=sum1, b=c, sum=sum,

carry=carry2); Or(a=carry1, b=carry2,

out=carry);

}
```



2.3 Add16 (16-bit Adder)

Adds two 16-bit numbers using a ripple-carry chain of FullAdders. The most significant carry-out is ignored (two's complement wrap-around).

```
// Add16.hdl // Implements: out = a + b (16-bit two's

complement addition)

CHIP Add16 {

IN a[16], b[16];

OUT out[16];
```

```
PARTS:

HalfAdder(a=a[0], b=b[0], sum=out[0], carry=c0);

FullAdder(a=a[1], b=b[1], c=c0, sum=out[1], carry=c1);

FullAdder(a=a[2], b=b[2], c=c1, sum=out[2], carry=c2);

FullAdder(a=a[3], b=b[3], c=c2, sum=out[3], carry=c3);

FullAdder(a=a[4], b=b[4], c=c3, sum=out[4], carry=c4);

FullAdder(a=a[5], b=b[5], c=c4, sum=out[5], carry=c5);

FullAdder(a=a[6], b=b[6], c=c5, sum=out[6], carry=c6);

FullAdder(a=a[7], b=b[7], c=c6, sum=out[7], carry=c7);

FullAdder(a=a[8], b=b[8], c=c7, sum=out[8], carry=c8);

FullAdder(a=a[9], b=b[9], c=c8, sum=out[9], carry=c9);

FullAdder(a=a[10], b=b[10], c=c9, sum=out[10], carry=c10);

FullAdder(a=a[11], b=b[11], c=c10, sum=out[11], carry=c11);

FullAdder(a=a[12], b=b[12], c=c11, sum=out[12], carry=c12);

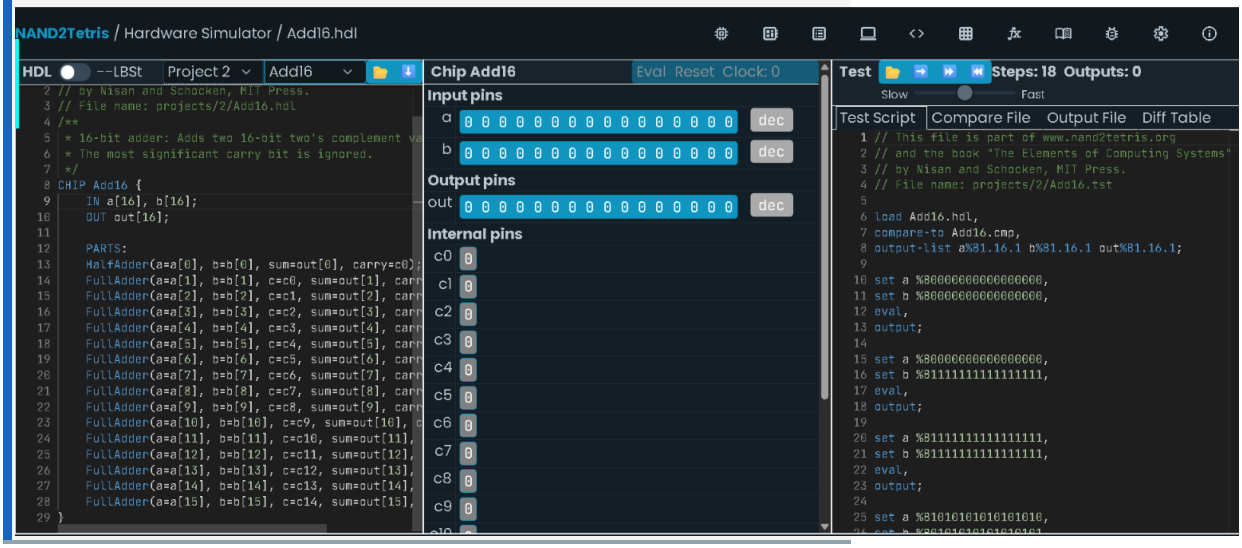
FullAdder(a=a[13], b=b[13], c=c12, sum=out[13], carry=c13);

FullAdder(a=a[14], b=b[14], c=c13, sum=out[14], carry=c14);

FullAdder(a=a[15], b=b[15], c=c14, sum=out[15], carry=c15);

// c15 (overflow carry) is intentionally discarded

}
```



2.4 ALU (Arithmetic Logic Unit)

The Hack ALU computes one of 18 functions of two 16-bit inputs (x and y) based on six control bits: **zx, nx, zy, ny, f, no**. It also outputs two status bits: **zr** (output is zero) and **ng** (output is negative).

Control Bit Functions:

Bit	Effect when set to 1
zx	Zero the x input

nx	Negate the x input (bitwise NOT)
zy	Zero the y input
ny	Negate the y input (bitwise NOT)
f	If 1: compute x+y; If 0: compute x AND y
no	Negate the output (bitwise NOT)

```
// ALU.hdl // The Hack ALU — 18 possible
functions of x and y

CHIP ALU { IN
x[16], y[16], zx,
nx, zy, ny, f, no;
OUT out[16], zr,
ng;

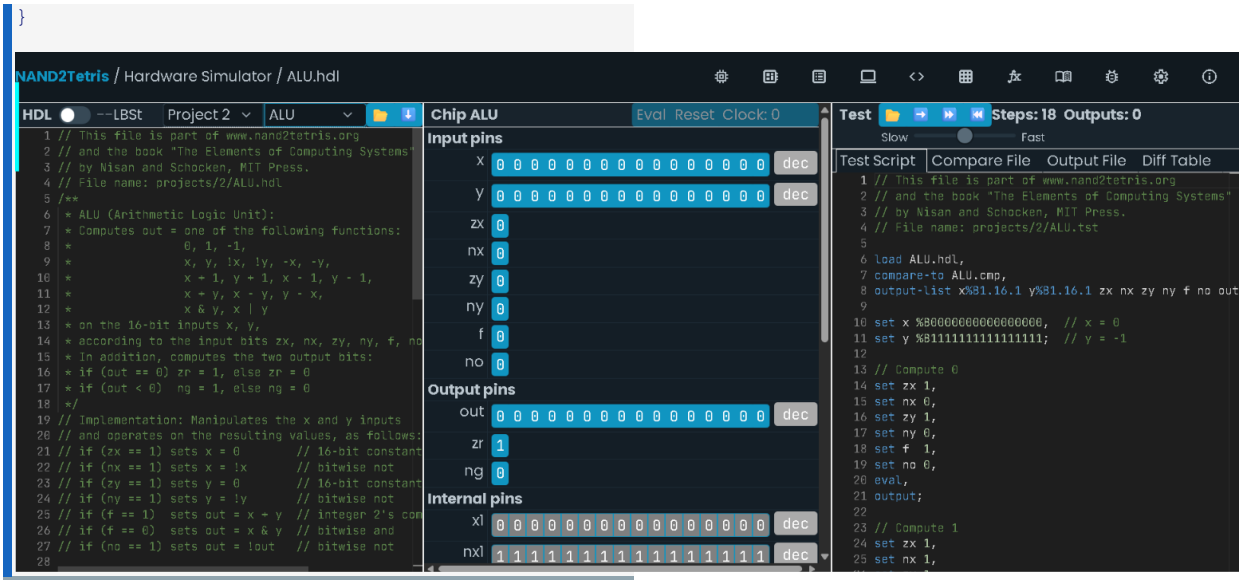
PARTS:
// --- Pre-process x --Mux16(a=x, b=false, sel=zx,
out=x1); // zx: zero x?
Not16(in=x1, out=notX1); Mux16(a=x1, b=notX1, sel=nx,
out=x2); // nx: negate x?

// --- Pre-process y --Mux16(a=y, b=false, sel=zy,
out=y1); // zy: zero y?
Not16(in=y1, out=notY1); Mux16(a=y1, b=notY1, sel=ny,
out=y2); // ny: negate y?

// --- Compute f(x2, y2) ---
And16(a=x2, b=y2, out=andXY);
Add16(a=x2, b=y2, out=addXY); Mux16(a=andXY, b=addXY,
sel=f, out=fOut); // f: AND or ADD?

// --- Post-negate output ---
Not16(in=fOut, out=notFOut); Mux16(a=fOut, b=notFOut, sel=no,
out=out, out[0..7]=lo, out[8..14]=hi, out[15]=ng); // ng: MSB
= sign bit
```

```
// --- Compute zr (out == 0?) ---
Or8Way(in=lo, out=orLo);
Or8Way(in[0..6]=hi, in[7]=false, out=orHi);
Or(a=orLo, b=orHi, out=anyBit);
Not(in=anyBit, out=zr);
```


}
NAND2Tetris / Hardware Simulator / ALU.hdl
HDL --LBSt Project 2 ALU
Chip ALU Eval Reset Clock: 0
Input pins
x 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 dec
y 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 dec
zx 0
nx 0
zy 0
ny 0
f 0
no 0
Output pins
out 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 dec
zr 1
ng 0
Internal pins
x1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 dec
nx1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 dec
Test Steps: 18 Outputs: 0
Slow Fast
Test Script Compare File Output File Diff Table
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/2/ALU.tst
load ALU.hdl,
compare-to ALU.cmp,
output-list x%B1.16.1 y%B1.16.1 zx nx zy ny f no out
set x %B0000000000000000, // x = 0
set y %B1111111111111111; // y = -1
// Compute 0
set zx 1,
set nx 0,
set zy 1,
set ny 0,
set f 1,
set no 0,
eval,
output;
// Compute 1
set zx 1,
set nx 1,